

Hamiltonian Cycles through Two Capacity-Bounded Connected Blocks

A composed reconstruction • Awaiting expert review
2026-09-22

Abstract

We give an explicit polynomial-time reduction from Hamiltonian Cycle to partitioning a weighted graph into at most two connected blocks of bounded weight. Positional Boolean clauses encode a cyclic ordering, carry-free integer digits encode their satisfiability, and two weighted anchor vertices turn exact subset sum into connected partitioning. Every valid target partition recovers a Hamiltonian cycle; a valid negative answer also recovers correctly. The construction uses binary integers and has polynomial bit complexity, including empty and small source graphs. It composes standard reduction ingredients and makes no claim of a new complexity classification or practical solver advantage.

1. Introduction

Connected vertex partitions do not generally describe spanning paths. A spanning tree can branch, and its path between two prescribed endpoints need not visit all vertices. This distinction obstructs the converse in a proposed reduction from Hamiltonian Cycle to bounded-component spanning forest [1]. A correct reconstruction must encode cyclic traversal through additional constraints.

We use numerical capacity to express those constraints. Hamiltonian SAT encodings are established [2], and the variable-and-clause digit construction for SAT to Subset Sum is standard [3]. The contribution here is a self-contained composition with executable recovery from every valid target output. The target graph is bipartite and has only two anchors. Its connected blocks may be stars; no step interprets a block tree as a path.

2. Preliminaries

The source is a simple undirected graph on an explicit list of distinct integer labels $u_0, \dots, u_{n-1}$. A valid output lists every vertex once in cyclic order, with every consecutive pair and the closing pair adjacent. Simple undirected cycles have at least three vertices. If no such cycle exists, the valid output is NO-SOLUTION.

The target consists of a simple graph, nonnegative integer vertex weights, and positive integers K and B . A valid output partitions all vertices into at most K nonempty connected blocks, each of weight at most B . A block may additionally carry a spanning-tree certificate. If no partition exists, the target output is NO-SOLUTION. Integers are binary encoded; graph vertices and edges are explicit. Empty target graphs admit the empty partition.

Theorem 1 (main result). There are deterministic polynomial-time maps F and G such that, for every source graph x and every valid target output y of $F(x)$, $G(x, y)$ is a valid Hamiltonian Cycle output. For $n \geq 3$, the target has $K = 2$ and is a complete bipartite graph with one side of size two. The forward encoding has $O(n^6 \log^2 n)$ bits.

3. Construction

For $n < 3$, output a single vertex of weight two with $K = B = 1$. This target is infeasible, as is the source. We now define the nontrivial branch, with $n \geq 3$.

3.1. Positional clauses

Fix u_0 as the cycle root and put $r = n - 1$. For remaining-vertex and position indices $v, p \in \{0, \dots, r - 1\}$, introduce $X_{v,p}$ to mean that u_{v+1} occurs in position p after the root. Require exactly one true variable

for every position and exactly one for every remaining vertex. Each requirement consists of the positive clause on its r variables and all pairwise negative clauses.

If u_{v+1} is not adjacent to u_0 , add the unit clauses $\neg X_{v,0}$ and $\neg X_{v,r-1}$. For each $p < r - 1$ and ordered pair of distinct nonadjacent remaining vertices v, w , add

$$\neg X_{v,p} \vee \neg X_{w,p+1}.$$

There are $V = r^2$ variables. Number $X_{v,p}$ by $i = vr + p$, and let C be the number of clauses. Every clause is nonempty, uses distinct variables, and has width at most r . The clause and literal counts are $O(n^3)$.

3.2. Carry-free integer items

Let clause j have width w_j . Define its slack ceiling and the radix by

$$h_j = 2^{\lceil \log_2 w_j \rceil}, \quad R = 2 \max_j h_j.$$

Unit clauses have $h_j = 1$. Because the formula includes width- r positive clauses, $R \geq 4$. Use base- R digit positions $0, \dots, V - 1$ for variables and $V, \dots, V + C - 1$ for clauses.

For each variable, create its true item and false item in that order. Both have digit one at that variable's position and zero at other variable positions. At clause position $V + j$, a truth item's digit is one precisely when that choice satisfies a literal in clause j . All other clause digits are zero.

For clause j , append slack items whose only nonzero digits, at position $V + j$, are the powers of two below h_j . There are no slack items for unit clauses. The desired sum T has digit one at every variable position and digit h_j at clause position $V + j$. Order the items with all true/false pairs first, followed by slack items in clause order and increasing binary weight. Write the resulting positive integers as $a_1, \dots, a_M$, and let $S = \sum_{i=1}^M a_i$.

3.3. Two weighted anchors

The graph in Figure 1 has anchors α and β with weights S and $2T$. Each item becomes a vertex of weight a_i , adjacent to both anchors and to no other vertex. There is no anchor-anchor edge. Set

$$K = 2, \quad B = S + T.$$

The graph has $M + 2$ vertices and $2M$ edges.

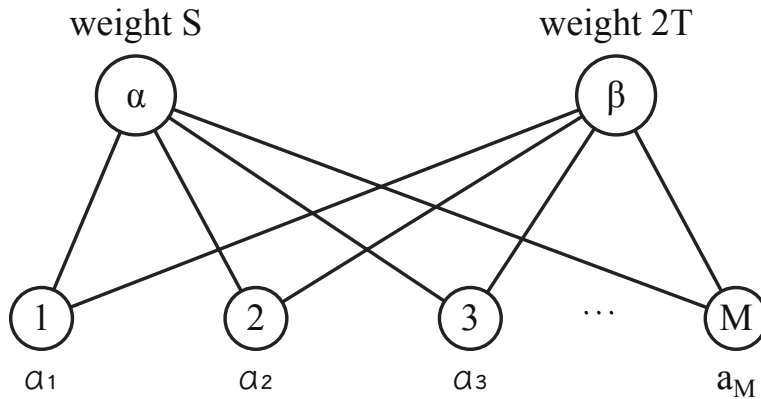


Figure 1: The target graph $K_{2,M}$. Item vertices 1, 2, 3 and M are shown; each omitted item represented by the ellipsis is also adjacent to both anchors. All eight edges incident to the displayed items are drawn; there are no other edge types. Item weights appear below their vertices. Total weight $2B$ forces two full blocks, and the anchors' combined weight exceeds B .

4. Correctness

Lemma 1 (ordering). The positional formula is satisfiable exactly when the source has a Hamiltonian cycle. Every satisfying assignment gives such a cycle starting at u_0 .

Proof. The row and column exactly-one clauses give a permutation of all remaining vertices. The nonedge clauses require every consecutive edge, and the unit clauses require both edges incident to the root. Conversely, any Hamiltonian cycle can be rotated to begin at u_0 and then satisfies these clauses. Both orientations are allowed. $\square$

Lemma 2 (digits). A subset of the items sums to T exactly when its truth choices give a satisfying assignment with suitable slack. Every subset of sum T gives such an assignment.

Proof. Even the sum of all items has no carry. Its variable digits equal two, less than R . The full sum at clause digit j is

$$w_j + (h_j - 1) \leq 2h_j - 1 < R. \quad (1)$$

Every subset has no larger digit sums. Thus equality to T is equivalent to equality at each digit. Variable digits force exactly one of the two truth items.

Let t_j count the satisfied literals in clause j . Binary slack realizes exactly the integers from zero to $h_j - 1$. Since $0 \leq t_j \leq w_j \leq h_j$, the clause digit can be completed to h_j precisely when $t_j \geq 1$. This proves both directions, including unit clauses. $\square$

The constructed parameters satisfy $S > T > 0$. All items are positive because every truth item has a variable digit and every slack item has a positive clause digit. The full-item sum has variable digits two instead of one; its clause digits $w_j + h_j - 1$ are at least h_j . By the absence of carries, this coordinatewise domination is strict as an integer comparison.

Lemma 3 (anchors). Every feasible target partition separates the anchors and selects item sum T in the block containing α . Conversely, any subset of item sum T gives a feasible partition.

Proof. The total vertex weight is $2S + 2T = 2B$. At most two blocks, each of weight at most B , must therefore mean exactly two blocks, each of weight B . The anchors cannot share a block, because

$$S + 2T = B + T > B.$$

The block containing α consequently has item weight $B - S = T$. Conversely, put any subset of sum T with α , and put the other items with β . Both blocks have weight B and are connected through their anchors. $\square$

4.1. Recovery for every output

Number target anchors zero and one, then number the item vertices from two. The true-choice item for variable i has vertex index $2 + 2i$. Given a partition, G finds the block containing vertex zero. For each position p , it selects the unique v whose vertex $2 + 2(vr + p)$ is in that block, then returns u_0 followed by the selected vertices in position order. It ignores optional tree certificates. Block and vertex order do not affect this procedure.

Proof of Theorem 1. Lemmas 3, 2 and 1 show that every feasible target partition decodes to a Hamiltonian cycle. In the other direction, a cycle yields a satisfying assignment, a subset of sum T , and a feasible target partition. Thus feasibility is equivalent. The decoder maps NO-SOLUTION to the same answer, which is valid exactly when the source is infeasible. For $n < 3$, the overweight singleton already supplies that equivalence. Both endpoints always have a witness or their designated negative answer. The construction and recovery are deterministic and have the bounds below. $\square$

5. Complexity

The intermediate formula has $O(n^3)$ digit positions. Each clause contributes at most $O(\log n)$ slack items, so

$$M = O(n^3 \log n), \quad \log R = O(\log n).$$

Each item and T therefore has $O(n^3 \log n)$ bits. Summing the items or doubling T adds only $O(\log M)$ bits. This proves the stated total target encoding bound, including its explicit edge list. Integer values may be exponentially large; their bit lengths are polynomial. No weight is expanded into a unary graph gadget or used as a loop bound.

Let L denote source bit length and H denote supplied target-output bit length. Explicit vertices ensure $n = O(L)$. With school arithmetic and integer conversion, the implementation fits the conservative bounds $O((L + 1)^{15})$ bit operations for F and $O((L + 1)^8)$ output bits. These deliberately loose bounds include exponentiation, weight sums, label comparisons and worst-case hash collisions. They are polynomiality guarantees, not optimized runtime estimates.

Recovery constructs only clause counts, scans and sorts partition indices, and checks the true-item positions. It performs no arithmetic on target weights and no search over assignments. Its conservative bit-operation bound is $O((L + H + 1)^6)$, and its returned cycle uses $O(L + 1)$ bits. Arbitrary signed source labels and large optional certificates are covered by these input lengths. The implementation disables fixed integer-text digit ceilings in both command modes.

6. Conclusion

The composition supplies a complete reduction to the specified connected-partition endpoint using two anchors and binary capacities. Every-output recovery follows from capacity saturation and digit equality, without a path assumption on connected blocks. This result does not establish strong hardness, a bounded-numeric-weight construction, a new complexity classification, or efficient target solving. Earliest priority for the full composition remains unassessed; no novelty claim is based on that coverage gap.

References

- [1] CodingThrust/problem-reductions. *Hamiltonian Circuit to Bounded Component Spanning Forest*, issue 238. GitHub issue. Accessed 2026-09-22. Used as the reconstruction request, not as a proof source.
- [2] Marijn J. H. Heule. *Chinese Remainder Encoding for Hamiltonian Cycles*. Theory and Practice of Satisfiability Testing, SAT 2021, LNCS 12831, pp. 216–224. Author manuscript. The introduction provides encoding context; its particular construction is not used.
- [3] Xanda Schoefield and Eva Tardos. *Subset Sum is NP-complete*. Cornell CS4820 lecture notes, 29 October 2018, Theorem 1 and Claims 2–3, pp. 1–3. Course notes. The present proof uses binary slack instead of repeated unit slack.

A. Verification and reproducibility

An independent verifier executes both maps as subprocesses, enumerates source cycles, and solves the actual target instances. Its two-block solver uses Boolean membership and unsigned bit-vector sums. The width is the bit length of the sum of all nonnegative weights, so no subset or partial sum can overflow. Direct graph traversal checks connectivity; disconnected assignments are blocked and search continues. Negative answers require conclusive exhaustion, and unknown results raise execution errors.

The completed finite domain contains every simple graph on at most four labeled vertices, a five-cycle, a five-vertex articulation graph, and path/triangle cases with signed 5,001-digit labels. All 80 instances passed: 13 source YES and 67 NO. There were 97 valid target outputs and 127 recovery calls, including reversed block and vertex order and optional spanning-tree certificates. The largest target had 122 vertices and 288-bit weights and capacity. The oracle requests at most twelve valid partitions per instance; the general proof supplies all-output coverage.

A registered independent review added exhaustive checks of both triangle and path targets. It examined all 131,072 two-block assignments for each target, finding two valid triangle partitions and none for the path. Recovery passed for each witness and the negative answer. The prepared general- K label/depth oracle passed nine source and seven target self-test fixtures, then all nine prepared candidate instances with seventeen recovery calls. Its bounded labels, depths and capacity sums use exact unsigned bit-vector encodings; the connectivity constraints remain independent of the additional verifier's graph-traversal filtering. Historical interrupted executions are retained in the round records, never used as negative answers. These finite checks neither replace the proof nor establish large-instance solver performance.

The locked environment uses CPython 3.14.2, uv 0.12.7 and z3-solver 5.1.0.0 (Z3 reports 5.1.0). Compilation uses Typst 0.15.1; PDF inspection uses Poppler 26.09.0. From the repository root:

```
uv sync --locked
cd campaigns/hamiltonian-cycle-bounded-component-forest/work
uv run --no-sync python check.py --self-test
uv run --no-sync python check.py --candidate algorithm.py
uv run --no-sync python verify.py --candidate algorithm.py
uv run --no-sync python ../reviews/initial/check.py
typst compile manuscript.typ manuscript.pdf
```

The forward command is `python algorithm.py`; recovery is `python algorithm.py --extract`. Both read one JSON value from standard input. Recovery expects source and target_solution; target outputs use blocks and optionally trees. Exact encodings and retained execution evidence are in contract.md, preparation.md, verification.md and the round records. The scripts have finite instance/output bounds and no solver or subprocess timeouts. No sibling repository is a runtime dependency.